Optimization for Pretraining

pig7selene

September 5, 2026

Abstract

Pretraining minimizes a token-level likelihood objective, but the objective alone does not specify a viable training procedure. This chapter develops the optimization machinery that turns the loss of Chapter 5 into parameter updates: mini-batch gradients, momentum, Adam, AdamW, learning-rate control, batch sizing, gradient accumulation, clipping, and optimizer-state accounting. The central theme is that an optimizer configuration is a coupled system. Its update count, valid-token count, learning-rate schedule, regularization, and numerical contracts must refer to the same training event.

1 Introduction

Chapter 5 expressed Causal Language Modeling as the minimization of token-level negative log-likelihood, and Chapter 6 described how a data pipeline supplies the valid target tokens on which that loss is measured. Optimization closes the loop: it maps an estimate of the loss gradient to a change in the parameters. The mapping is consequential. At pretraining scale, a small mistake in loss normalization, update counting, learning-rate scheduling, or optimizer-state restoration can alter every subsequent update.

Let z denote a valid target-token training event, including the context that predicts it, and let $\ell(\theta; z)$ be its negative log-likelihood under parameters θ. The population objective is

$$\mathcal{J}(\theta) = \mathrm{E}_{z \sim P}[\ell(\theta; z)], \tag{1}$$

where P is the effective data distribution induced by the mixture and sequence-construction policy of Chapter 6. The full expectation is unavailable, so each optimizer update uses a finite collection of valid token events. This chapter assumes that gradients of the decoder-only Transformer have already been obtained by backpropagation. The questions are how to combine them, scale them, regularize the resulting update, and account for the state required to repeat the procedure.

2 Mini-Batch Optimization

At update t, let $\mathcal{B}_t$ contain n_t valid target-token events. The mini-batch gradient is

$$g_t = \frac{1}{n_t} \sum_{z \in \mathcal{B}_t} \nabla_\theta \ell(\theta_t; z). \tag{2}$$

Here n_t is the number of loss-eligible tokens, not necessarily the number of sequences in device memory. Padding, document-boundary policies, and loss masks can make those quantities differ substantially. The implementation should therefore use the same valid-token convention for the loss reported in Chapter 5 and for the gradient that drives the update.

If $\mathcal{B}_t$ is sampled from P under the stated mixture policy, g_t is an estimate of $\nabla \mathcal{J}(\theta_t)$. Its randomness is useful: it makes optimization feasible without processing the whole corpus before every update. It also means that a training run is governed by a stochastic process rather than by a deterministic descent curve. Changing the number of valid tokens per update changes both the computational work and the distribution of this estimate.

Plain Stochastic Gradient Descent (SGD) applies

$$\theta_{t+1} = \theta_t - \eta_t g_t, \tag{3}$$

where $\eta_t > 0$ is the learning rate. This equation remains the reference point for more elaborate optimizers. Every additional mechanism can be understood as changing the direction g_t, the scale η_t, or both.

3 SGD and Momentum

The gradient from one mini-batch can fluctuate because token events are a limited and heterogeneous sample. Momentum reduces the influence of fluctuations that reverse from update to update while preserving directions that recur. With a velocity u_t and momentum coefficient μ, a common form is

$$u_t = \mu u_{t-1} + g_t, \quad \theta_{t+1} = \theta_t - \eta_t u_t, \quad 0 \le \mu < 1. \tag{4}$$

Expanding the recurrence shows that u_t is an exponentially weighted history of gradients. A coherent direction is reinforced across updates; a rapidly alternating component tends to cancel. The exact momentum convention varies across software libraries, including whether the learning rate is placed inside the velocity. What matters is to identify the convention before transferring hyperparameters. Momentum has long been central to practical deep-network optimization, although its success depends on the interaction among initialization, curvature, and the momentum schedule [1].

SGD with momentum uses one scalar learning rate for every parameter coordinate. That can be effective, but Transformer pretraining contains parameters with different gradient scales and nonstationary statistics. An adaptive optimizer addresses this by maintaining coordinate-wise estimates of gradient scale.

4 Adam and AdamW

4.1 First and Second Moment Estimates

Adam maintains two state tensors for each optimized parameter tensor. The first moment m_t is an exponential moving average of the gradient, while the second moment v_t is an exponential moving average of its elementwise square:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2. \tag{5}$$

The square in Equation 5 is elementwise. Thus m_t resembles momentum, whereas v_t records a recent uncentered second moment for each coordinate. Dividing by the square root of v_t makes a coordinate with persistently large gradients receive a smaller normalized step than one with small gradients. Adam was introduced as a first-order method based on adaptive estimates of these moments [2].

The state tensors are usually initialized at zero. Consequently, their early values are biased toward zero, particularly when β_1 or β_2 is close to one. Adam corrects this initialization bias by defining

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}. \tag{6}$$

The adaptive update is then

$$\theta_{t+1} = \theta_t - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}, \tag{7}$$

where division and the square root are elementwise, and $\varepsilon > 0$ protects the denominator. The pair (β_1, β_2) sets the memory of the two statistics; ε is part of the update rule rather than an incidental implementation detail.

Adam is common in language-model pretraining not because it removes optimization choices, but because it combines gradient smoothing with coordinate-wise scaling. This is attractive when a deep model has heterogeneous parameter groups, when gradient scales change during early training, and when a global SGD learning rate would be difficult to tune. Its state is also simple to serialize and its cost per parameter is predictable. The price of this adaptivity is additional memory and additional hyperparameters, discussed in Section 8 and Section 9.

4.2 Decoupled Weight Decay

L2 regularization adds a penalty $\lambda \|\theta\|_2^2$ to the objective. Its gradient adds $\lambda\theta_t$ to the data gradient. With plain SGD, the resulting update is

$$\theta_{t+1} = \theta_t - \eta_t(g_t + \lambda\theta_t) = (1 - \eta_t\lambda)\theta_t - \eta_t g_t. \tag{8}$$

This algebra explains why L2 regularization and weight decay are often treated as equivalent in SGD: both shrink parameters by the scalar factor $1 - \eta_t\lambda$. The equivalence does not survive Adam's coordinate-wise preconditioner. Let D_t be the diagonal operator whose jth diagonal entry is

$$(D_t)_{j,j} = \frac{1}{\sqrt{\hat{v}_{t,j}} + \varepsilon}. \tag{9}$$

Coupling the L2 penalty to Adam then gives $\theta_{t+1} = \theta_t - \eta_t D_t(g_t + \lambda\theta_t)$. The shrinkage is scaled by D_t and therefore differs across coordinates.

AdamW instead applies the decay independently of the adaptive data-gradient update:

$$\theta_{t+1} = (1 - \eta_t\lambda)\theta_t - \eta_t D_t g_t. \tag{10}$$

This is decoupled Weight Decay. It restores a direct, coordinate-independent shrinkage step while retaining Adam's adaptive update for the data loss. Loshchilov and Hutter formalized the distinction and showed that treating L2 regularization as Weight Decay in adaptive methods is misleading [3]. In practice, a parameter-group policy must state which tensors decay. It is common to exempt bias and normalization parameters, but that is a design choice to record and validate, not a consequence of the AdamW equation.

5 Learning-Rate Control

The learning rate determines the size of an optimizer step after all other transformations. In AdamW, it appears in both the adaptive update and the decoupled shrinkage factor. A schedule is therefore a time-dependent definition of the optimizer, not a cosmetic post-processing operation.

5.1 Warmup

Warmup starts with a small learning rate and increases it over an initial interval of W optimizer updates. Linear warmup to a peak value $\eta_{\max}$ is

$$\eta_t = \eta_{\max} \min\left(1, \frac{t}{W}\right). \tag{11}$$

At the start of training, random initialization, incomplete moment estimates, and rapidly changing activation statistics make a peak learning rate unnecessarily risky. A large early update can move parameters into a regime from which the loss recovers slowly or not at all. Warmup limits this transient while the model and optimizer establish their initial scales. Large-mini-batch experiments demonstrated that an early-training warmup can resolve optimization difficulties that arise when scaling the learning rate with batch size [4]; studies of Transformer normalization also connect warmup sensitivity to the scale of early gradients [5].

Warmup is not a substitute for a viable peak learning rate. It changes the beginning of the trajectory, whereas the peak value, optimizer coefficients, batch size, and model parameterization continue to determine its subsequent behavior. Its duration should be stated in optimizer updates and, for cross-run comparison, in valid tokens seen.

5.2 Schedules

After warmup, a schedule usually holds or reduces the learning rate as training progresses. One common decay is a cosine transition from $\eta_{\max}$ to $\eta_{\min}$ over updates $W \le t \le U$:

$$\eta_t = \eta_{\min} + \frac{\eta_{\max} - \eta_{\min}}{2} \times \left(1 + \cos\left(\pi \frac{t - W}{U - W}\right)\right). \tag{12}$$

Linear decay, constant-with-decay, and other schedules are also used. No formula is inherently canonical; the schedule must be specified together with its clock. A schedule indexed by updates changes

meaning when gradient accumulation changes the number of optimizer steps per token. A schedule indexed by valid tokens avoids that ambiguity but still requires an explicit mapping from tokens to update boundaries. Compute-optimal language-model studies accordingly treat the learning-rate schedule as an experimental variable coupled to model and token budgets [6].

6 Batch Size, Gradient Noise, and Accumulation

6.1 Batch Size and Gradient Noise

Under an idealized independent-sampling model, the conditional variance of the mini-batch estimate scales approximately as

$$\mathrm{Var}[g_t \texttt{ mid } \theta_t] \approx \frac{\Sigma(\theta_t)}{n_t}, \tag{13}$$

where $\Sigma(\theta_t)$ is the covariance associated with one valid target-token event. Language-model tokens within a sequence are correlated, and mixture sampling introduces further structure, so this equation is a guide rather than an exact accounting identity. Its central implication remains useful: a larger effective batch usually reduces the stochastic variation of the update estimate.

This variation is called gradient noise. It is not simply an error term. Moderate noise can help exploration of a nonconvex objective, while excessive noise makes the loss trajectory erratic and can trigger instability. As batch size grows, the marginal reduction in noise eventually becomes small compared with the extra work required to form an even larger batch. Empirical work on large-batch training uses the gradient noise scale to characterize this trade-off [7].

Learning rate and batch size must be tuned together. Increasing the effective batch often permits a larger learning rate because the gradient estimate is less variable, but a linear scaling rule is a useful hypothesis, not a universal law. It depends on the optimizer, the data distribution, the schedule, and the regime of the model. When batch size changes, warmup duration, total updates, schedule clock, and clipping rate are quantities to revisit rather than constants to inherit blindly.

6.2 Gradient Accumulation

The physical batch is the amount of data whose forward and backward pass fits at one time. When it is too small to reach a desired effective batch, gradient accumulation processes K micro-batches without updating the parameters, sums their gradients, and takes one optimizer step. For micro-batch k with n_k valid target tokens, the desired aggregate is

$$g_t = \frac{1}{N} \sum_{k=1}^{K} \sum_{z \in \mathcal{B}_{t,k}} \nabla_\theta \ell(\theta_t; z), \quad N = \sum_{k=1}^{K} n_k. \tag{14}$$

Thus an implementation must weight each micro-batch by its valid-token count. Averaging K already-averaged losses is wrong when the numbers of valid tokens differ. Accumulation changes the effective batch without storing all activations simultaneously, but it is not identical to increasing the physical batch in every respect. It requires K separate forward and backward passes, has a different reduction order and runtime profile, and can differ when a model includes batch-dependent operations or stochastic behavior that is not controlled consistently. It also changes the number of optimizer steps per token unless the surrounding schedule is adjusted.

The decisive contract is that AdamW updates m_t, v_t, the learning-rate clock, and Weight Decay once after the aggregate gradient has been formed—not once per micro-batch. Applying any of these per micro-batch creates a different optimizer.

7 Gradient Clipping

Gradient clipping limits the influence of an unusually large update direction. Global-norm clipping replaces g_t by

$$\tilde{g}_t = g_t \min\left(1, \frac{c}{\|g_t\|_2}\right), \tag{15}$$

where c is a configured threshold. Gradients below the threshold are unchanged; a gradient above it is rescaled without changing direction. The technique was introduced as a practical response to exploding gradients [8], and it remains a useful guardrail for rare loss spikes or abnormal batches during pretraining.

Clipping does not repair a chronically unsuitable learning rate, corrupted data, or an incorrect loss reduction. A high clipping frequency is diagnostic information. Moreover, clipping each micro-batch before accumulation differs from clipping the aggregate gradient in Equation 14. The latter corresponds to the effective update whose norm is being limited and should be the default meaning unless the implementation explicitly chooses another policy.

8 Optimizer State and Memory Cost

For P optimized scalar parameters, AdamW stores a first-moment tensor and a second-moment tensor with the same logical shape as the parameters. If each state element occupies b_s bytes, the moment states alone require approximately

$$M_{\text{optimizer}} \approx 2Pb_s. \tag{16}$$

This excludes the parameters themselves, gradients, and any optional master-parameter copy. If the parameter, gradient, and moment-state representations occupy b_w, b_g, and b_s bytes per scalar respectively, a useful model-side lower-bound accounting is

$$M_{\text{parameters-plus-states}} \approx P\big(b_w + b_g + 2b_s\big). \tag{17}$$

For example, half-precision parameters and gradients with full-precision moment states consume $2P + 2P + 4P + 4P = 12P$ bytes before any master copy, activations, temporary buffers, or distributed-training considerations. The exact layout is framework- and precision-policy-dependent, but the qualitative conclusion is not: optimizer state is a first-class memory cost. A checkpoint that omits m_t, v_t, parameter-group settings, or the update index cannot resume the same AdamW trajectory.

9 Hyperparameter Interactions

The most important pretraining hyperparameters form coupled groups rather than an independent checklist. Table Table 1 summarizes the interactions that should be reasoned about together.

Configuration change	Immediate effect	Required accompanying check
Larger effective batch	Less variable gradient estimate and fewer updates per token at fixed token budget	Retune learning rate and warmup; state whether schedule is indexed by updates or valid tokens.
More accumulation steps	Larger effective batch without larger activation residency	Normalize by total valid tokens; step AdamW, decay, and scheduler only after accumulation.
Higher peak learning rate	Larger adaptive parameter updates	Revisit warmup, clipping frequency, decay strength, and loss-spike monitoring.
Different β_1 or β_2	Different time scale for gradient statistics	Reset and checkpoint moment states consistently; do not compare schedules by learning rate alone.
Different Weight Decay	Different parameter shrinkage over updates	Check the learning-rate schedule, decay exclusions, and total number of optimizer steps.

Table 1: Optimization settings that must be interpreted jointly. Each row changes the meaning of at least one other control.

The accumulation example is especially easy to miss. If total training is specified in tokens and the effective batch doubles, the number of optimizer updates is approximately halved. A warmup defined as a fixed number of updates then covers roughly twice as many tokens; a cosine decay defined over

a fixed update count no longer ends at the intended token budget. The experiment should choose one primary clock and derive the other explicitly.

10 Implementation Contracts

The loss-reduction contract must preserve the objective in Chapter 5. For variable-length micro-batches, accumulate the **sum** of loss over valid target tokens and divide by the total valid-token count N of the accumulation cycle. Padding tokens, ignored labels, and masked positions must contribute neither to this numerator nor denominator. The logged loss, gradient normalization, and token counter should agree on the same mask.

The optimizer-step contract is equally strict. Compute all micro-batch gradients at fixed θ_t, form the aggregate gradient, apply global-norm clipping if configured, update AdamW's moments and bias-correction index once, apply its decoupled decay once, and advance the learning-rate schedule once. Clearing gradients or stepping the scheduler inside the accumulation loop violates this contract.

The parameter-group contract should enumerate the learning rate, (β_1, β_2), ε, Weight Decay, and decay-exclusion policy for every group. The checkpoint contract should preserve parameters, both moment tensors, update index, scheduler state, parameter-group configuration, and counters for valid tokens and samples. Finally, monitoring should record at least loss, learning rate, global gradient norm before clipping, clipping events, update count, and valid tokens seen. These traces make an optimization failure diagnosable rather than anecdotal.

11 Summary

Pretraining optimization estimates the gradient of a token-level likelihood objective with mini-batches and maps that estimate to parameter updates. SGD supplies the basic update, momentum smooths a history of gradients, and Adam combines first-moment smoothing with coordinate-wise scaling from second moments. AdamW separates adaptive data-gradient updates from Weight Decay, avoiding the false equivalence between L2 regularization and decay under an adaptive preconditioner.

Learning rate, warmup, schedule, effective batch size, and gradient accumulation define the timescale and stochasticity of training together. Gradient clipping bounds exceptional update norms, while AdamW's moment tensors make optimizer state a substantial memory and checkpointing concern. A reliable pretraining run is therefore not defined by an optimizer name alone: it is defined by a coherent set of equations, clocks, normalization rules, state tensors, and auditable implementation contracts.

References

[1] I. Sutskever, J. Martens, G. Dahl, and G. Hinton, "On the Importance of Initialization and Momentum in Deep Learning," in *Proceedings of the 30th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 28. PMLR, 2013, pp. 1139–1147. [Online]. Available: https://proceedings.mlr.press/v28/sutskever13.html

[2] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *International Conference on Learning Representations*, 2015. doi: 10.48550/arXiv.1412.6980.

[3] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in *International Conference on Learning Representations*, 2019. [Online]. Available: https://openreview.net/forum?id=Bkg6RiCqY7

[4] P. Goyal *et al.*, "Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour," *arXiv preprint arXiv:1706.02677*, 2017, doi: 10.48550/arXiv.1706.02677.

[5] R. Xiong *et al.*, "On Layer Normalization in the Transformer Architecture," in *Proceedings of the 37th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 10524–10533.

[6] J. Hoffmann *et al.*, "Training Compute-Optimal Large Language Models," *arXiv preprint arXiv:2203.15556*, 2022, [Online]. Available: https://arxiv.org/abs/2203.15556

[7] S. McCandlish, J. Kaplan, D. Amodei, and O. D. Team, "An Empirical Model of Large-Batch Training," *arXiv preprint arXiv:1812.06162*, 2018, doi: 10.48550/arXiv.1812.06162.

[8] R. Pascanu, T. Mikolov, and Y. Bengio, "On the Difficulty of Training Recurrent Neural Networks," in *Proceedings of the 30th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 28. PMLR, 2013, pp. 1310–1318. [Online]. Available: https://proceedings.mlr.press/v28/pascanu13.html